Centrifuge: Onchain PM

March 2026

security review

reviewed by:

xmxanuel

cantina.xyz/u/xmxanuel

15th April 2026

1. Executive Summary

The Centrifuge team has asked xmxanuel to review the Executor module of their Solidity codebase as an independent security reviewer. The scope includes the Executor contract, Weiroll VM integration, Guards (ApprovalGuard, SlippageGuard, CircuitBreakerGuard), and ExecutorHelpers.

The Executor is a Weiroll VM-based execution engine that allows strategists to run pre-approved scripts (Merkle-leaf authorized) against pool assets. Scripts are sequences of external calls with a shared state array. Three guard contracts provide runtime bounds: ApprovalGuard (ERC20 allowance cleanup), SlippageGuard (bookend value-change limits), and CircuitBreakerGuard (rolling-window throughput/deviation limits).

2. Disclaimer

This security review report is provided "as is" and for informational purposes only. The purpose of this report is to assist the client in identifying potential security vulnerabilities in the reviewed code, based on the scope and methodology agreed upon. It does not constitute legal, investment, financial, or any other form of professional advice.

No warranties or guarantees are made regarding the completeness, accuracy, or security of the reviewed code. The author and any affiliated parties make no representations or warranties, express or implied, and expressly disclaim any liability or responsibility for any loss, damage, or other consequence arising from or related to the use of, reliance on, or inability to use this report or the reviewed code. This includes, without limitation, direct, indirect, incidental, consequential, special, exemplary, or punitive damages, even if advised of the possibility of such damages.

This report reflects the results of a best-effort security review conducted within the scope and time constraints agreed upon with the client. Although an attempt was made to identify potential security issues, there is no assurance that all vulnerabilities have been discovered. A smart contract security review cannot be considered a guarantee or certification of security. Users and project teams remain solely responsible for the use, deployment, and management of any smart contracts.

This report does not constitute an endorsement or recommendation of the reviewed project. Third parties should not rely on this report and are strongly encouraged to perform their own independent assessments.

3. Repository

https://github.com/centrifuge/protocol-internal/

Version	Commit Hash	Date	Note
Review Version	02ee5df1	March 2026	review version

Scope

- `src-ir/Executor.sol` Weiroll VM-based execution engine
- `src/managers/spoke/ExecutorHelpers.sol` Helper library (addBps, subBps)
- `src/managers/spoke/guards/ApprovalGuard.sol` ERC20 allowance cleanup
- `src/managers/spoke/guards/SlippageGuard.sol` Value-change limits
- `src/managers/spoke/guards/CircuitBreakerGuard.sol` Rolling-window throughput limits
- `lib/enso-weiroll/contracts/` Weiroll VM and CommandBuilder

4. Findings Overview

The following findings were identified during the security review of the codebase.

Summary

Severity	ID	Title	Status
Low	L1	Callback slot consumption by untrusted protocol	Fixed
Low	L2	Unchecked uint128 downcast in CircuitBreakerGuard.delta()	Fixed
Low	L3	Unchecked uint128 downcast in SlippageGuard._trackPeriodLoss()	Fixed
Low	L4	Missing bps validation in ExecutorHelpers.addBps()	Fixed
Low	L5	CircuitBreakerGuard tally window is a free parameter	Fixed
Low	L6	No on-chain enforcement of write-protected state slots	Fixed
Informational	I1	Weiroll `0xFE` (IDX_USE_STATE) full state overwrite	Acknowledged
Informational	I2	FLAG_DATA allows calling any function on approved targets	Acknowledged
Informational	I3	Weiroll writeTuple lacks out-of-bounds check	Acknowledged
Informational	I4	Dirty memory reads from incorrect array lengths in state	Acknowledged
Informational	I5	Callback subscript state is isolated	Acknowledged
Informational	I6	Native ETH management, leftover ETH shared across strategists	Acknowledged
Informational	I7	Uniswap V3 multi-hop routes require separate scripts per route length	Acknowledged
Informational	I8	Dynamic state slot content not validated by Weiroll	Acknowledged
Informational	I9	Leaf verification complexity	Acknowledged

5. Low Findings

L1. Callback slot consumption by untrusted protocol

Executor.sol#L111

Callbacks were consumed sequentially by index with no binding between the callback hash and the contract expected to trigger it. An untrusted protocol interacted with earlier in the script could call `executeCallback` and consume a callback slot meant for a later caller. Normally this causes a revert when the legitimate caller arrives. However, if the legitimate protocol conditionally skips its callback (e.g. Morpho skips when `data` is empty), the consumed count still matches `callbackHashes.length` and execution succeeds with the callback having executed in the wrong context.

Recommendation: Bind each callback to its expected `msg.sender`.

Centrifuge: Fixed in 4cc7f71e. `Callback` struct now includes a `caller` address. `executeCallback` verifies `msg.sender == callbacks[idx].caller`.

xmxanuel: Fixed.

L2. Unchecked uint128 downcast in CircuitBreakerGuard.delta()

CircuitBreakerGuard.sol#L40

`s.anchor = uint128(currentValue)` performed an unchecked truncation from `uint256`. Values above `type(uint128).max` would silently wrap, producing an incorrect anchor and allowing deviations that should have been rejected. This was inconsistent with `tally()` which already used `toUint128()`.

Recommendation: Use safe cast `currentValue.toUint128()`.

Centrifuge: Fixed in cd72c12b.

xmxanuel: Fixed.

L3. Unchecked uint128 downcast in SlippageGuard._trackPeriodLoss()

SlippageGuard.sol#L141

`_trackPeriodLoss(poolId, scId, uint128(loss))` truncated a `uint256` loss value. A loss exceeding `type(uint128).max` would wrap to a small number, bypassing the period loss limit.

Recommendation: Use safe cast `loss.toUint128()`.

Centrifuge: Fixed in cd72c12b.

xmxanuel: Fixed.

L4. Missing bps validation in ExecutorHelpers.addBps()

ExecutorHelpers.sol#L69

`addBps()` did not validate that `bps <= BPS_BASE`, unlike `subBps()` which did. The inconsistency could lead to unexpected results if the caller assumed the same validation as `subBps`.

Recommendation: Add `require(bps <= BPS_BASE, InvalidBps())` to match `subBps`.

Centrifuge: Fixed in cd72c12b.

xmxanuel: Fixed.

L5. CircuitBreakerGuard tally window is a free parameter

CircuitBreakerGuard.sol#L19

The `tally()` function accepted `window` as a call parameter. If a script passed a shorter window for the same key, it could produce an earlier accumulator reset, effectively bypassing the rate limit.

Recommendation: Include `window` in the mapping key to prevent window manipulation.

Centrifuge: Fixed in 8e2677d4. `window` is now part of the mapping key: `mapping(caller => mapping(key => mapping(window => CumulativeState)))`.

xmxanuel: Fixed.

L6. No on-chain enforcement of write-protected state slots

State slots intended to be fixed (governance-approved values) could be overwritten by Weiroll commands that write their output into those slots. A malicious script could modify a fixed parameter mid-execution and reset it before the script ends, making it undetectable.

Two mitigation approaches were discussed:

1. **Pre/post hash** hash bitmap-marked slots before and after `_execute`, revert on mismatch. Simple and gas-efficient but vulnerable to modify-then-reset within the script.
2. **Command output pointer iteration** iterate all commands and verify no output pointer targets a fixed slot. Stronger but higher gas cost.

Centrifuge: Fixed in a0aa4a97 and c62fa9d6. Option 1 implemented via `stateBitmap` and `_hashBitmapSlots` pre/post comparison at Executor.sol#L97-99. The modify-then-reset weakness was acknowledged as acceptable given the same level of trust required for script approval.

xmxanuel: Fixed.

6. Informational Findings

I1. Weiroll `0xFE` (IDX_USE_STATE) full state overwrite

CommandBuilder.sol#L457

The Weiroll output flag `IDX_USE_STATE` replaces the entire state array with the return value of a command. A malicious script could overwrite all state slots, including write-protected ones (the bitmap check runs after `_execute` completes, so modify-then-reset is possible). This is hard to spot in raw command bytes during leaf verification.

Centrifuge: Acknowledged. The off-chain TypeScript SDK that generates scripts from JSON workflows will never emit this flag. Anyone can reproduce the SDK output to verify a policy hash.

xmxanuel: Acknowledged.

I2. FLAG_DATA allows calling any function on approved targets

VM.sol#L15

When `FLAG_DATA` is used, the Weiroll command sends a state slot as raw calldata. Since the state slot is freely chosen by the strategist, this allows calling any function selector on an approved target. If the target has a callback mechanism (e.g. Morpho), the callback destination is also strategist-controlled.

Centrifuge: Acknowledged. Off-chain SDK validation ensures only expected patterns are used.

xmxanuel: Acknowledged.

I3. Weiroll writeTuple lacks out-of-bounds check

CommandBuilder.sol#L495

`writeTuple` does not perform the same bounds check as `writeOutputs`. Additionally, it does not reject special index values like `IDX_USE_STATE` (`0xFE`). It would write to state slot 126 (`0xFE & 0x7F`). In practice this still reverts (state arrays are limited to 128 slots), but the failure mode is less explicit.

Centrifuge: Acknowledged.

xmxanuel: Acknowledged.

I4. Dirty memory reads from incorrect array lengths in state

If a strategist provides a state slot with an array whose declared length exceeds the actual number of encoded elements, the target contract's Solidity ABI decoder reads into adjacent memory. For example, `myFunc(uint256[], uint256[])` where `arr1` has `length=3` but only 2 elements, the decoder reads the `length` field of `arr2` as `arr1[2]`.

Not exploitable because: (a) the strategist could have provided any arbitrary value directly, and (b) Solidity's ABI decoder handles this predictably. Non-Solidity targets may behave differently.

Centrifuge: Acknowledged.

xmxanuel: Acknowledged.

I5. Callback subscript state is isolated

Executor.sol#L111

Each callback invocation receives its own state array. Output writes in the callback subscript are not propagated back to the main script's state. This is inherent to the callback architecture (separate `executeCallback` calls with independent state).

Off-chain tooling should warn script authors that callback side-effects on state are not visible to the parent script.

Centrifuge: Acknowledged, by design.

xmxanuel: Acknowledged.

I6. Native ETH management, leftover ETH shared across strategists

Executor.sol#L44

The Executor's `receive()` accepts ETH, and leftover ETH (e.g. from overpaying cross-chain gas) remains in the contract. Any strategist with an approved script can access this ETH via `VALUECALL`. There is no per-strategist ETH separation.

Acceptable for the current trust model where all strategists for a pool are equally trusted. If strategist trust levels diverge in the future, a post-execution ETH refund mechanism should be considered.

Centrifuge: Acknowledged.

xmxanuel: Acknowledged.

I7. Uniswap V3 multi-hop routes require separate scripts per route length

Protocols using per-step callbacks (e.g. Uniswap V3 pool-level swaps) require the callback count to be known at script approval time. A 2-pool route and a 3-pool route need separate approved scripts.

Mitigation: use peripheral routers (e.g. Uniswap SwapRouter) that abstract multi-hop routing into a single call.

Centrifuge: Acknowledged, design limitation.

xmxanuel: Acknowledged.

I8. Dynamic state slot content not validated by Weiroll

Weiroll copies raw bytes from state slots without validating their internal ABI structure. A dynamic-typed slot used as an array parameter could contain a mismatched length header. Solidity's decoder on the target contract tolerates trailing bytes, but non-Solidity implementations may not.

Centrifuge: Acknowledged.

xmxanuel: Acknowledged.

I9. Leaf verification complexity

Verifying a Weiroll script leaf (commands + state) requires deep understanding of the VM's encoding: output flags, dynamic types, tuple operations, special indices. This makes manual auditing of individual leaves impractical. If an untrusted party constructs leaves and a trusted party must verify them, the verification burden is high.

Centrifuge: Acknowledged. The planned off-chain SDK provides a deterministic JSON-to-commands pipeline. Verifiers reproduce the build from the JSON source rather than auditing raw command bytes.

xmxanuel: Acknowledged.